26-Context容器（下）：Tomcat如何实现Servlet规范？

我们知道，Servlet容器最重要的任务就是创建Servlet的实例并且调用Servlet，在前面两期我谈到了Tomcat如何定义自己的类加载器来加载Servlet，但加载Servlet的类不等于创建Servlet的实例，类加载只是第一步，类加载好了才能创建类的实例，也就是说Tomcat先加载Servlet的类，然后在Java堆上创建了一个Servlet实例。

一个Web应用里往往有多个Servlet，而在Tomcat中一个Web应用对应一个Context容器，也就是说一个Context容器需要管理多个Servlet实例。但Context容器并不直接持有Servlet实例，而是通过子容器Wrapper来管理Servlet，你可以把Wrapper容器看作是Servlet的包装。

那为什么需要Wrapper呢？Context容器直接维护一个Servlet数组不就行了吗？这是因为Servlet不仅仅是一个类实例，它还有相关的配置信息，比如它的URL映射、它的初始化参数，因此设计出了一个包装器，把Servlet本身和它相关的数据包起来，没错，这就是面向对象的思想。

那管理好Servlet就完事大吉了吗？别忘了Servlet还有两个兄弟：Listener和Filter，它们也是Servlet规范中的重要成员，因此Tomcat也需要创建它们的实例，也需要在合适的时机去调用它们的方法。

说了那么多，下面我们就来聊一聊是Tomcat如何做到上面这些事的。

Servlet管理

前面提到，Tomcat是用Wrapper容器来管理Servlet的，那Wrapper容器具体长什么样子呢？我们先来看看它里面有哪些关键的成员变量：

```
protected volatile Servlet instance = null;
```

毫无悬念，它拥有一个Servlet实例，并且Wrapper通过loadServlet方法来实例化Servlet。为了方便你阅读，我简化了代码：

```
public synchronized Servlet loadServlet() throws ServletException {
    Servlet servlet;

    //1. 创建一个Servlet实例
    servlet = (Servlet) instanceManager.newInstance(servletClass);

    //2.调用了Servlet的init方法，这是Servlet规范要求的
    initServlet(servlet);

    return servlet;
}
```

其实loadServlet主要做了两件事：创建Servlet的实例，并且调用Servlet的init方法，因为这是Servlet规范要求的。

仅供个人学习 请勿传播 用户标记 mannix小童鞋

那接下来的问题是，什么时候会调到这个loadServlet方法呢？为了加快系统的启动速度，我们往往会采取资源延迟加载的策略，Tomcat也不例外，默认情况下Tomcat在启动时不会加载你的Servlet，除非你把Servlet的`loadOnStartup`参数设置为`true`。

这里还需要你注意的是，虽然Tomcat在启动时不会创建Servlet实例，但是会创建Wrapper容器，就好比尽管枪里面还没有子弹，先把枪造出来。那子弹什么时候造呢？是真正需要开枪的时候，也就是说有请求来访问某个Servlet时，这个Servlet的实例才会被创建。

那Servlet是被谁调用的呢？我们回忆一下专栏前面提到过Tomcat的Pipeline-Valve机制，每个容器组件都有自己的Pipeline，每个Pipeline中有一个Valve链，并且每个容器组件有一个BasicValve（基础阀）。Wrapper作为一个容器组件，它也有自己的Pipeline和BasicValve，Wrapper的BasicValve叫**StandardWrapperValve**。

你可以想到，当请求到来时，Context容器的BasicValve会调用Wrapper容器中Pipeline中的第一个Valve，然后会调用到StandardWrapperValve。我们先来看看它的invoke方法是如何实现的，同样为了方便你阅读，我简化了代码：

```
public final void invoke(Request request, Response response) {

    //1.实例化Servlet
    servlet = wrapper.allocate();

    //2.给当前请求创建一个Filter链
    ApplicationFilterChain filterChain =
        ApplicationFilterFactory.createFilterChain(request, wrapper, servlet);

   //3. 调用这个Filter链，Filter链中的最后一个Filter会调用Servlet
   filterChain.doFilter(request.getRequest(), response.getResponse());

}
```

StandardWrapperValve的invoke方法比较复杂，去掉其他异常处理的一些细节，本质上就是三步：

- 第一步，创建Servlet实例；
- 第二步，给当前请求创建一个Filter链；
- 第三步，调用这个Filter链。

你可能会问，为什么需要给每个请求创建一个Filter链？这是因为每个请求的请求路径都不一样，而Filter都有相应的路径映射，因此不是所有的Filter都需要来处理当前的请求，我们需要根据请求的路径来选择特定的一些Filter来处理。

第二个问题是，为什么没有看到调到Servlet的service方法？这是因为Filter链的doFilter方法会负责调用Servlet，具体来说就是Filter链中的最后一个Filter会负责调用Servlet。

接下来我们来看Filter的实现原理。

Filter管理

我们知道，跟Servlet一样，Filter也可以在web.xml文件里进行配置，不同的是，Filter的作用域是整个Web应用，因此Filter的实例是在Context容器中进行管理的，Context容器用Map集合来保存Filter。

```
private Map<String, FilterDef> filterDefs = new HashMap<>();
```

那上面提到的Filter链又是什么呢？Filter链的存活期很短，它是跟每个请求对应的。一个新的请求来了，就动态创建一个FIlter链，请求处理完了，Filter链也就被回收了。理解它的原理也非常关键，我们还是来看看源码：

```
public final class ApplicationFilterChain implements FilterChain {

  //Filter链中有Filter数组，这个好理解
  private ApplicationFilterConfig[] filters = new ApplicationFilterConfig[0];

  //Filter链中的当前的调用位置
  private int pos = 0;

  //总共有多少了Filter
  private int n = 0;

  //每个Filter链对应一个Servlet，也就是它要调用的Servlet
  private Servlet servlet = null;

  public void doFilter(ServletRequest req, ServletResponse res) {
        internalDoFilter(request,response);
  }

  private void internalDoFilter(ServletRequest req,
                                ServletResponse res){

    // 每个Filter链在内部维护了一个Filter数组
    if (pos < n) {
        ApplicationFilterConfig filterConfig = filters[pos++];
        Filter filter = filterConfig.getFilter();

        filter.doFilter(request, response, this);
        return;
    }

    servlet.service(request, response);

}
```

从ApplicationFilterChain的源码我们可以看到几个关键信息：

1. Filter链中除了有Filter对象的数组，还有一个整数变量pos，这个变量用来记录当前被调用的Filter在数组中的位置。
2. Filter链中有个Servlet实例，这个好理解，因为上面提到了，每个Filter链最后都会调到一个Servlet。
3. Filter链本身也实现了doFilter方法，直接调用了一个内部方法internalDoFilter。
4. internalDoFilter方法的实现比较有意思，它做了一个判断，如果当前Filter的位置小于Filter数组的长度，也就是说Filter还没调完，就从Filter数组拿下一个Filter，调用它的doFilter方法。否则，意味着所有Filter

仅供个人学习 请勿传播 用户标记 mannix小童鞋

都调到了，就调用Servlet的service方法。

但问题是，方法体里没看到循环，谁在不停地调用Filter链的doFIlter方法呢？Filter是怎么依次调到的呢？

答案是**Filter本身的doFilter方法会调用Filter链的doFilter方法**，我们还是来看看代码就明白了：

```
public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain){

          ...

          //调用Filter的方法
          chain.doFilter(request, response);

      }
```

注意Filter的doFilter方法有个关键参数FilterChain，就是Filter链。并且每个Filter在实现doFilter时，必须要调用Filter链的doFilter方法，而Filter链中保存当前FIlter的位置，会调用下一个FIlter的doFilter方法，这样链式调用就完成了。

Filter链跟Tomcat的Pipeline-Valve本质都是责任链模式，但是在具体实现上稍有不同，你可以细细体会一下。

Listener管理

我们接着聊Servlet规范里Listener。跟Filter一样，Listener也是一种扩展机制，你可以监听容器内部发生的事件，主要有两类事件：

- 第一类是生命状态的变化，比如Context容器启动和停止、Session的创建和销毁。
- 第二类是属性的变化，比如Context容器某个属性值变了、Session的某个属性值变了以及新的请求来了等。

我们可以在web.xml配置或者通过注解的方式来添加监听器，在监听器里实现我们的业务逻辑。对于Tomcat来说，它需要读取配置文件，拿到监听器类的名字，实例化这些类，并且在合适的时机调用这些监听器的方法。

Tomcat是通过Context容器来管理这些监听器的。Context容器将两类事件分开来管理，分别用不同的集合来存放不同类型事件的监听器：

```
//监听属性值变化的监听器
private List<Object> applicationEventListenersList = new CopyOnWriteArrayList<>();

//监听生命事件的监听器
private Object applicationLifecycleListenersObjects[] = new Object[0];
```

剩下的事情就是触发监听器了，比如在Context容器的启动方法里，就触发了所有的ServletContextListener：

```
//1.拿到所有的生命周期监听器
Object instances[] = getApplicationLifecycleListeners();

for (int i = 0; i < instances.length; i++) {
   //2. 判断Listener的类型是不是ServletContextListener
   if (!(instances[i] instanceof ServletContextListener))
      continue;

   //3.触发Listener的方法
   ServletContextListener lr = (ServletContextListener) instances[i];
   lr.contextInitialized(event);
}
```

需要注意的是，这里的ServletContextListener接口是一种留给用户的扩展机制，用户可以实现这个接口来定义自己的监听器，监听Context容器的启停事件。Spring就是这么做的。ServletContextListener跟Tomcat自己的生命周期事件LifecycleListener是不同的。LifecycleListener定义在生命周期管理组件中，由基类LifeCycleBase统一管理。

本期精华

Servlet规范中最重要的就是Servlet、Filter和Listener“三兄弟”。Web容器最重要的职能就是把它们创建出来，并在适当的时候调用它们的方法。

Tomcat通过Wrapper容器来管理Servlet，Wrapper包装了Servlet本身以及相应的参数，这体现了面向对象中“封装”的设计原则。

Tomcat会给**每个请求生成一个Filter链**，Filter链中的最后一个Filter会负责调用Servlet的service方法。

对于Listener来说，我们可以定制自己的监听器来监听Tomcat内部发生的各种事件：包括Web应用级别的、Session级别的和请求级别的。Tomcat中的Context容器统一维护了这些监听器，并负责触发。

最后小结一下这3期内容，Context组件通过自定义类加载器来加载Web应用，并实现了Servlet规范，直接跟Web应用打交道，是一个核心的容器组件。也因此我用了很重的篇幅去讲解它，也非常建议你花点时间阅读一下它的源码。

课后思考

Context容器分别用了CopyOnWriteArrayList和对象数组来存储两种不同的监听器，为什么要这样设计，你可以思考一下背后的原因。

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。

仅供个人学习 请勿传播 用户标记 mannix小食鞋

极客时间

深入拆解 Tomcat & Jetty

从源码角度深度探索 Java 中间件



李号双

eBay 技术主管

新版升级：点击「请朋友读」，20位好友免费读，邀请订阅更有现金奖励。